Cycle Sort -

This sort describes an interesting approach to deal with problem involving arrays containing number in a given range, take the following problem:

> you are given an unsorted array containing number taken from the range 1 to 'n'. The array can have duplicates, which means that some number will be missing. Find all the missing number.

To efficiently solve this problem, we can use the fact that the input array contains in the range of 1 to 'n'. for example, to efficiently sort the array, we can trying each number in its correct place, i.e, placing '1' at index 0, placing '2' at index '1', and so on. Once we are done with sorting, we can iterate the array to find all indices that are missing the correct numbers.

Cyclic Sort →

We are given an array containing 'n' objects. Each object, when created, was assigned a unique number from 1 to 'n' based on their creation sequence. This means that the object with sequence number '3' was created just before the object with sequence number '4'.

Write a function to sort the objects in-place on their creation sequence number in $O(n)$ and without any extra space. For simplicity, let's assume we are passed an integer array containing only the sequence numbers, though each number is actually an object.

Example 1:

```
Input: [3, 1, 5, 4, 2]
Output: [1, 2, 3, 4, 5]
```

Example 2:

```
Input: [2, 6, 4, 3, 1, 5]
Output: [1, 2, 3, 4, 5, 6]
```

Example 3:

```
Input: [1, 5, 6, 4, 3, 2]
Output: [1, 2, 3, 4, 5, 6]
```

The input array contains numbers in the range of 1 to 'n'. We can use this fact to devise an efficient way to sort the numbers. Since all number are unique we can try placing each number at it's correct place, ie, place '1' at index '0', placing '2' at index '1', and so on.

To place a number (or an object in general) at its correct index, we first need to find that number. If we first find a number and then place it at it's correct place, it will take us $O(N^2)$, which is not acceptable.

Instead, what if we iterate the array one number at a time, and if the current number we are iterating is not at the correct index, we swap it with the number at it's correct index. This way we will go through all number and place them in their correct indices,

Number 2 is not at its correct place, let swap it with correct index.

We will not move on to the next number until we have a correct number at start

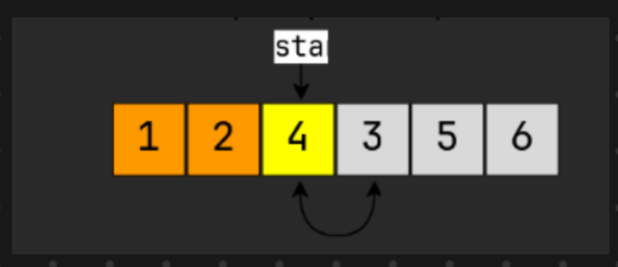
Number '6' is not at its correct place, lets swap it with the correct index

After the swap, both '6' and '1' are placed at their correct place.

Number '1' is at its correct index, lets move on the next number.

Number '2' is at its correct index, lets move on to the next number.

Number '4' is not at its correct place, lets swap it with its correct index.



After the swap, both '3' and '4' are placed at their correct place.

```cpp
#include<bits/stdc++.h>
using namespace std;

class CyclicSort {
 public:
    static void sort(vector<int>& nums){
        int i=0;
        while(i<nums.size()){
            int j = nums[i]-1;

            if(nums[i] != nums[j]){
                swap(num, i , j);

            }else{

                i++;
            }
        }
    }
 private:
    static void swap(vector<int>& arr, int i , int j){
        int temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;
    }

};
```

Problem - Find the missing Number (easy)

Problem Statement #

We are given an array containing 'n' distinct numbers taken from the range 0 to 'n'. Since the array has only 'n' numbers out of the total 'n+1' numbers, find the missing number.

Example 1:

```
Input: [4, 0, 3, 1]
Output: 2
```

Example 2:

```
Input: [8, 3, 5, 2, 4, 6, 0, 1]
Output: 7
```

1. In this problem, the number are ranged from '0' to 'n', compared to '1' to 'n' in the cyclic sort. This will make two changes in our Algorithm:
   - In this problem, each number should be equal to it index, compared to index + 1 in the Cyclic Sort. Therefore

$$nums[i] == nums[nums[i]]$$

   - Since the array will have 'n' numbers, which means array indices will range from 0 to 'n - 1'. Therefore, we will ignore the number 'n' as we can't place it in the array, so

$$nums[i] < nums.length$$

2) Say we are at index i. If we swap the number at index i to place it at the correct index, we can still have the wrong number at index i. This was true in cyclic sort too. It didn't cause any problem in cyclic sort as over there, we made sure to place one number at its correct place in each step, but that wouldn't be enough in this problem as we have one extra number due to the larger range. Therefore, we will not move to the next number after the swap until we have a correct number at the index i.

```
#include <bits/stdc++.h>
using namespace std;
```

```
class MissingNumber {
  public:
    static int findMissingNumber(vector<int>& nums){
      int i = 0;
      while( i < nums.size()){
        if(nums[i] < nums.size() && nums[i] != nums[nums[i]] ){
            swap(nums, i , nums[i]);
        }else {
            i++;
        }
      }
    // find the first number missing from its index, that will be our
       required number.
    for( i = 0; i < nums.size(); i++){
        if(nums[i] != i){
            return i;
        }
    }
    return nums.size();
  }
private:
    static void swap(vector<int>& arr, int i , int j ){
        int temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;
    }
};
```

Problem - Find all Missing Number

Problem Statement #

We are given an unsorted array containing numbers taken from the range 1 to 'n'. The array can have duplicates, which means some numbers will be missing. Find all those missing numbers.

Example 1:

```
Input: [2, 3, 1, 8, 2, 3, 5, 1]
Output: 4, 6, 7
Explanation: The array should have all numbers from 1 to 8, due to dupl
icates 4, 6, and 7 are missing.
```

Example 2:

```
Input: [2, 4, 1, 2]
Output: 3
```

```cpp
#include <bits/stdc++.h>
using namespace std;
class AllMissingNumbers{
public:
  static vector<int> findNumbers(vector<int>&nums){
    int i=0;
    while(i<nums.size()){
        if(nums[i]!=nums[nums[i]-1]){
            swap(nums,i,nums[i]-1);
        }else{
            i++;
        }
    }
    vector<int> missingNumber;
    for(i=0;i<nums.size();i++){
        if(nums[i]!=i+1){
            missingNumbers.push_back(i+1);
        }
    }
    return missingNumbers;
```

```
    }
private:
    static void swap(vector<int>&arr, int i , int j){
        int temp = arr[i];
        arr[i] = arr[j];
        arr[j] =  temp ;
    }
};
```

Find the Duplicate Number(easy)

We are given an unsorted array containing ‘n+1’ numbers taken from the range 1 to ‘n’. The array has only one duplicate but it can be repeated multiple times. **Find that duplicate number without using any extra space.** You are, however, allowed to modify the input array.

Example 1:

```
Input: [1, 4, 4, 3, 2]
Output: 4
```

Example 2:

```
Input: [2, 1, 3, 3, 5, 4]
Output: 3
```

```
#include<bits/stdc++.h>
using namespace std;
class FindDuplicate {
  public:
    static int findNumber(vector<int>& nums){
    int i=0;
    while(i<nums.size()){
        if(nums[i]!=i+1){
            if(nums[i]!=nums[nums[i]-1]){
                swap(nums,i,nums[i]-1);
            } else { // we have found the Duplicate.
                return nums[i];
            }
```

```
        } else {
            i++;
        }
    }
    return -1;
}
private:
    static void swap(vector<int>& arr, int i, int j){
        int temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;
    }
};
```

Similar Problem →

Can you solve the above problem in O(1) space & without modifying the input Array?

```
#include<bits/stdc++.h>
using namespace std;
class DuplicatesNumber{
    public:
        static int findDuplicate(const vector<int>& arr){
            int slow = 0;
            int fast = 0;
            do{
                slow = arr[slow];
                fast = arr[arr[fast]];
            }while(slow != fast);
```

```
        // find cycle length.
        int current = arr[slow];
        int cycleLength = 0;
        do {
            current = arr[current];
            cycleLength++;
        } while( current != arr[slow]);
        return findStart(arr, cycleLength);
    }
private:
    static int findStart(const vector<int> &arr, int cycleLength){
        int pointer1 = arr[0];
        int pointer2 = arr[0];
        // move pointer 2 ahead 'cyclelength' steps
        while(cycleLength > 0){
            pointer2 = arr[pointer2];
            cycleLength--;
        }
        // Increment both pointers until they meet at the start of the cycle.
        while (pointer1 != pointer2){
            pointer1 = arr[pointer1];
            pointer2 = arr[pointer2];
        }
        return pointer1;
    }
};
```

Problem - Find all Duplicate Numbers

We are given an unsorted array containing 'n' numbers taken from the range 1 to 'n'. The array has some duplicates, **find all the duplicate numbers without using any extra space.**

Example 1:

```
Input: [3, 4, 4, 5, 5]
Output: [4, 5]
```

Example 2:

```
Input: [5, 4, 7, 2, 3, 5, 3]
Output: [3, 5]
```

```cpp
#include <bits/stdc++.h>
using namespace std;
class FindAllDuplicate {
public:
  static vector<int> findNumbers(vector<int> &nums ){
    int i=0;
    while( i < nums.size()){
      if( nums[i] != nums[nums[i] - 1 ){
          swap(nums, i, nums[i] - 1);
      } else {
          i++;
      }
    }
  }
  vector<int> duplicateNumbers;
  for( i= 0; i < nums
```